

DATA-260 Sec 21&71 — Homework 2

Xu Wang

Repository: github.com/xuwang-goldenwater/data260-9275

Section 0 — Personal configuration and domain

Value	This submission	How it is derived
SID4	9275	last four digits of the SJSU Student ID
PORT_BASE	8275	$8000 + (9275 \bmod 900)$
PREFIX	s9275	"s" + SID4
SEED	9275	SID4
VERIFY_SEED	269275	$260000 + \text{SID4}$
DOMAIN_ID	3 — grocery supply and recall notices	$9275 \bmod 8$

Item	Value
Hardware	MacBook Pro, Apple silicon, 8 GB RAM, macOS
Local model	qwen3:8b via Ollama, temperature 0.0
Model adapter	src/model_client.py (unchanged from Homework 1)
Python	3.12, conda environment "data260"
Tagged commit	9096148
Tag	hw2

Homework 2 extends the Homework 1 codebase in place. Application code lives only in code/ and src/. reports/hw02/ holds evidence only. The Homework 1 self-check, verify_hw01.py, still passes without changes against the changed files.

Entity: RecallNotice, defined in DOMAIN_SCHEMA.md. Fields: productName (primary), recallingFirm (secondary), submitterEmail, description, category (four fixed values), agreedToTerms, submissionDate.

Part 1 — HTML & CSS

1.1 Usable at 375px

The Homework 1 page was one form in a fixed 640px column. Homework 2 adds a record list above it, a search bar, and two buttons on every record. All of that has to still work at 375px. That is the width of an iPhone SE.

Five rules do the work. Nothing else in styles.css sets a fixed width. `verify_hw02.py` checks for that.

- The stylesheet is mobile-first. The base rules are the narrow layout, and the one breakpoint widens it at 480px instead of narrowing it below.
- Every row that could overflow is a flex container with flex-wrap, so the search field and its two buttons stack instead of being squeezed.
- `min-width: 0` on the search input. Without it a flex item will not shrink below the width of its content, and it pushes the row wider than the screen.
- `overflow-wrap`: anywhere on product names and firm names. A long product name is one unbreakable token otherwise.
- Buttons are `min-height: 44px`, which is Apple's minimum comfortable touch target. The base font is 16px so iOS Safari does not zoom in when a field is focused.

code/web_application/styles.css — the toolbar row

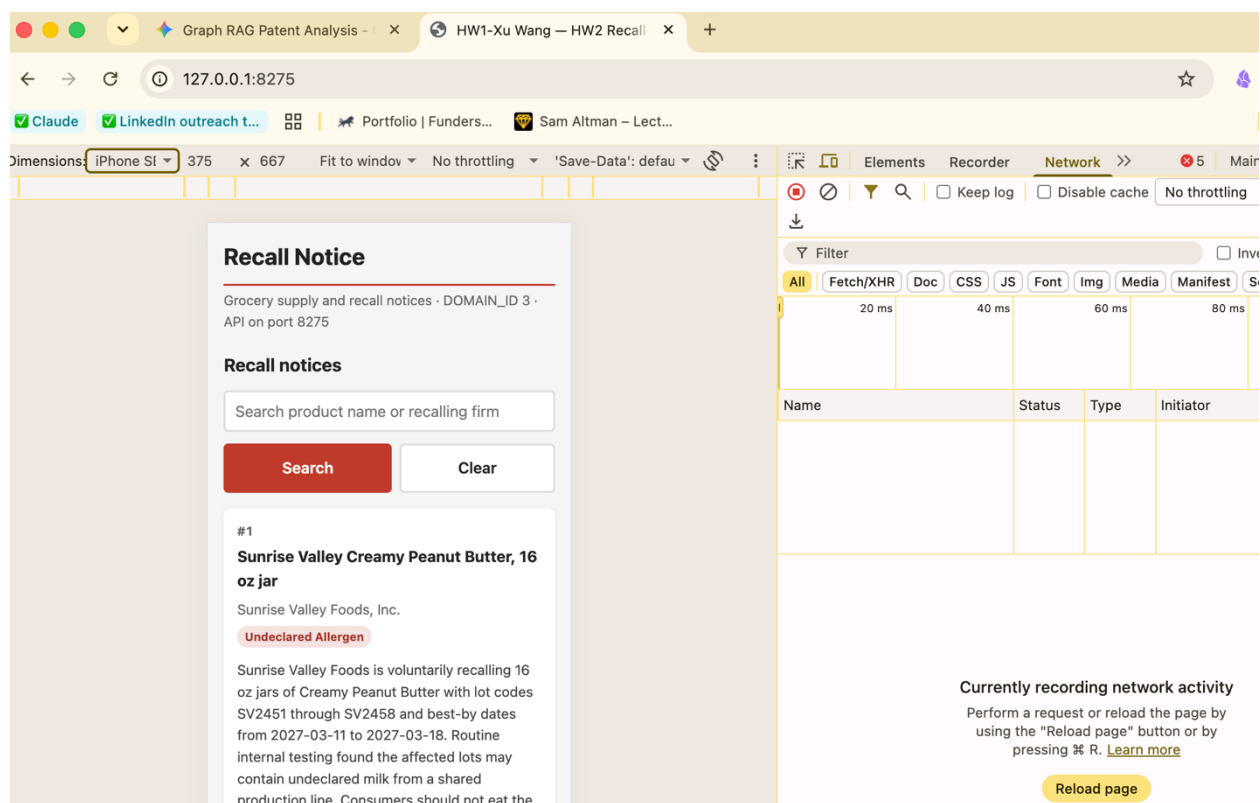
```
/* ----- toolbar: search + new ----- */
/* flex-wrap is what keeps this usable at 375px: the search field and the two
   buttons drop onto their own lines instead of being squeezed. */

.toolbar {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  margin-bottom: 16px;
}

.toolbar input[type="search"] {
  flex: 1 1 100%;
  min-width: 0;           /* without this a flex item refuses to shrink */
}

.toolbar button { flex: 1 1 auto; }

@media (min-width: 480px) {
  .toolbar input[type="search"] { flex: 1 1 60%; }
  .toolbar button { flex: 0 0 auto; }
}
```



1.2 Visible loading, empty and error states

All three states are written into the same element, #listState. Each function clears the list first, then replaces the class and the content of that element. Because they all write to the same place, two states cannot show at once. There is no combination of flags that can produce that.

code/web_application/app.js

```
// -----
const showLoading = () => {
  noticeList.innerHTML = "";
  listState.className = "state";
  listState.innerHTML = '<span class="spinner"></span>Loading recall notices&hellip;';
  listState.hidden = false;
};

const showEmpty = (query) => {
  noticeList.innerHTML = "";
  listState.className = "state";
  listState.textContent = query
    ? 'No recall notices match "' + query + '".'
    : "No recall notices on file yet. Use the button below to file the first one.";
```

```

    listState.hidden = false;
  });

const showError = (message) => {
  noticeList.innerHTML = "";
  listState.className = "state state-error";
  listState.innerHTML =
    "<strong>Could not load recall notices.</strong>" +
    "<span></span>" +
    '<button type="button" id="retryBtn">Retry</button>';
  listState.querySelector("span").textContent = message;
  listState.hidden = false;
  $("retryBtn").addEventListener("click", () => loadNotices(searchInput.value.trim()));
};

const hideState = () => { listState.hidden = true; };

```

code/web_application/styles.css

```

/* ----- loading / empty / error states ----- */

.state {
  background: var(--card);
  border-radius: var(--radius);
  padding: 24px 16px;
  text-align: center;
  color: var(--muted);
}

.state-error {
  background: #fdecea;
  color: #7f231c;
  border: 1px solid #f5c6c2;
  text-align: left;
}

.state-error strong { display: block; margin-bottom: 4px; color: var(--red-dark); }
.state-error button { margin-top: 10px; }

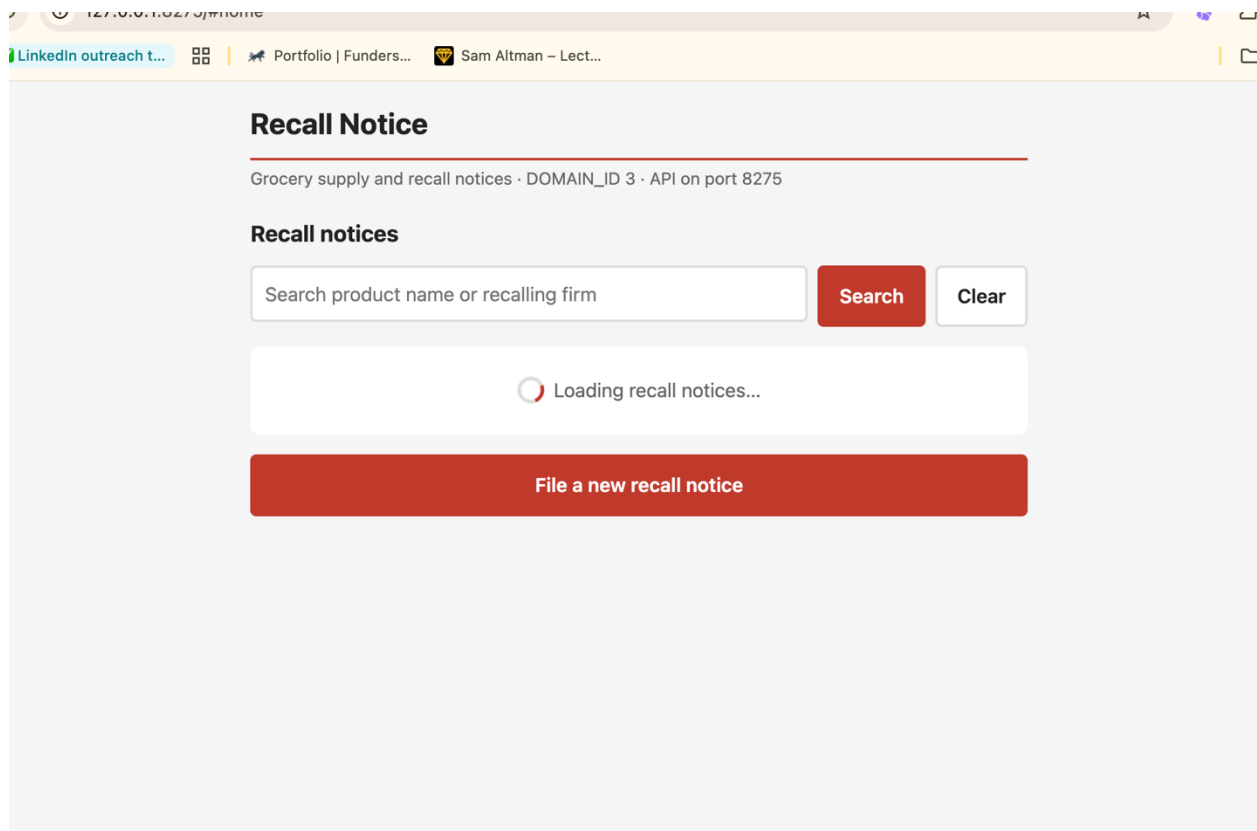
.spinner {
  display: inline-block;
  width: 22px;
  height: 22px;
  margin-right: 8px;
  vertical-align: -5px;
  border: 3px solid var(--line);
  border-top-color: var(--red);
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
}

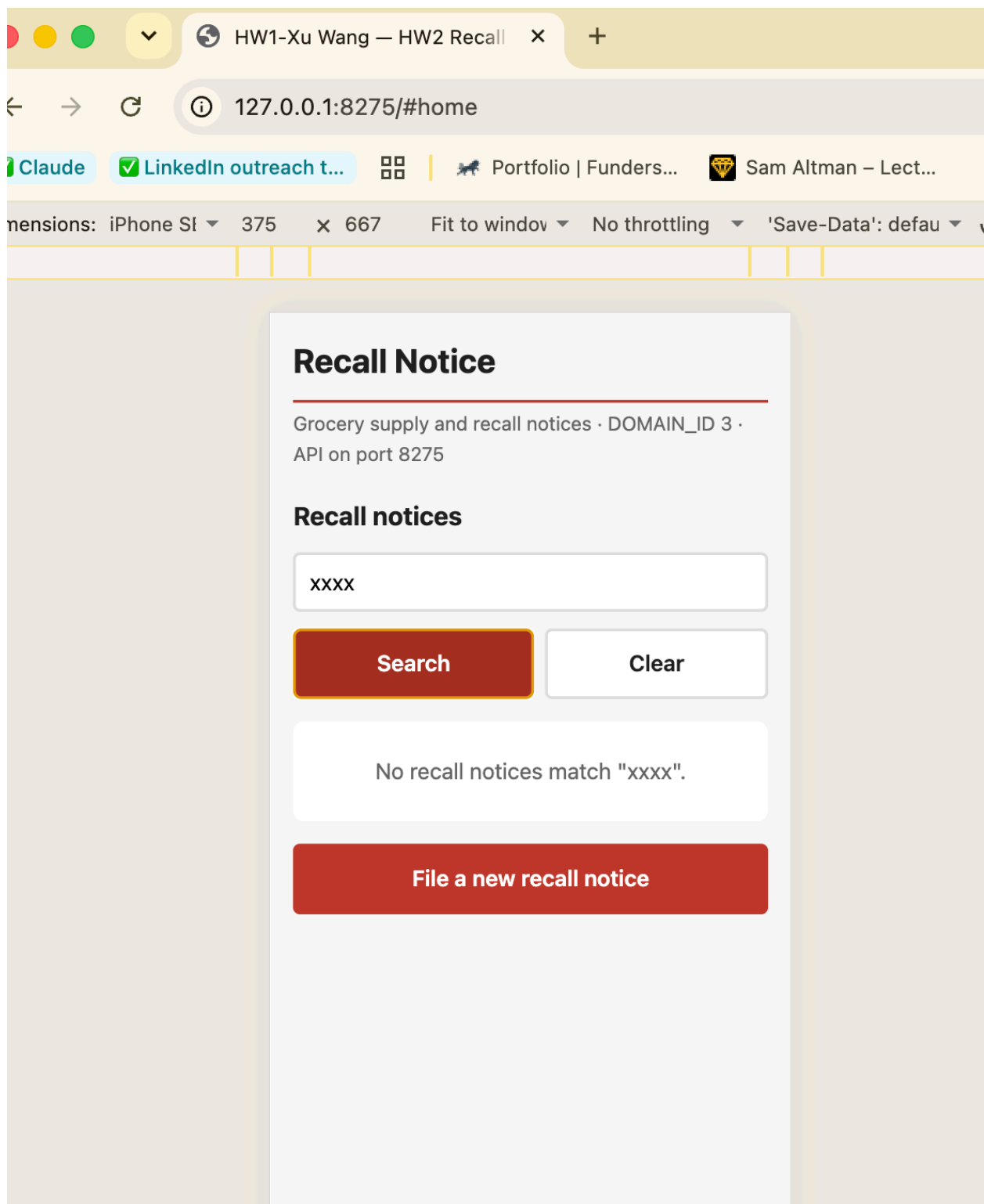
@keyframes spin { to { transform: rotate(360deg); } }

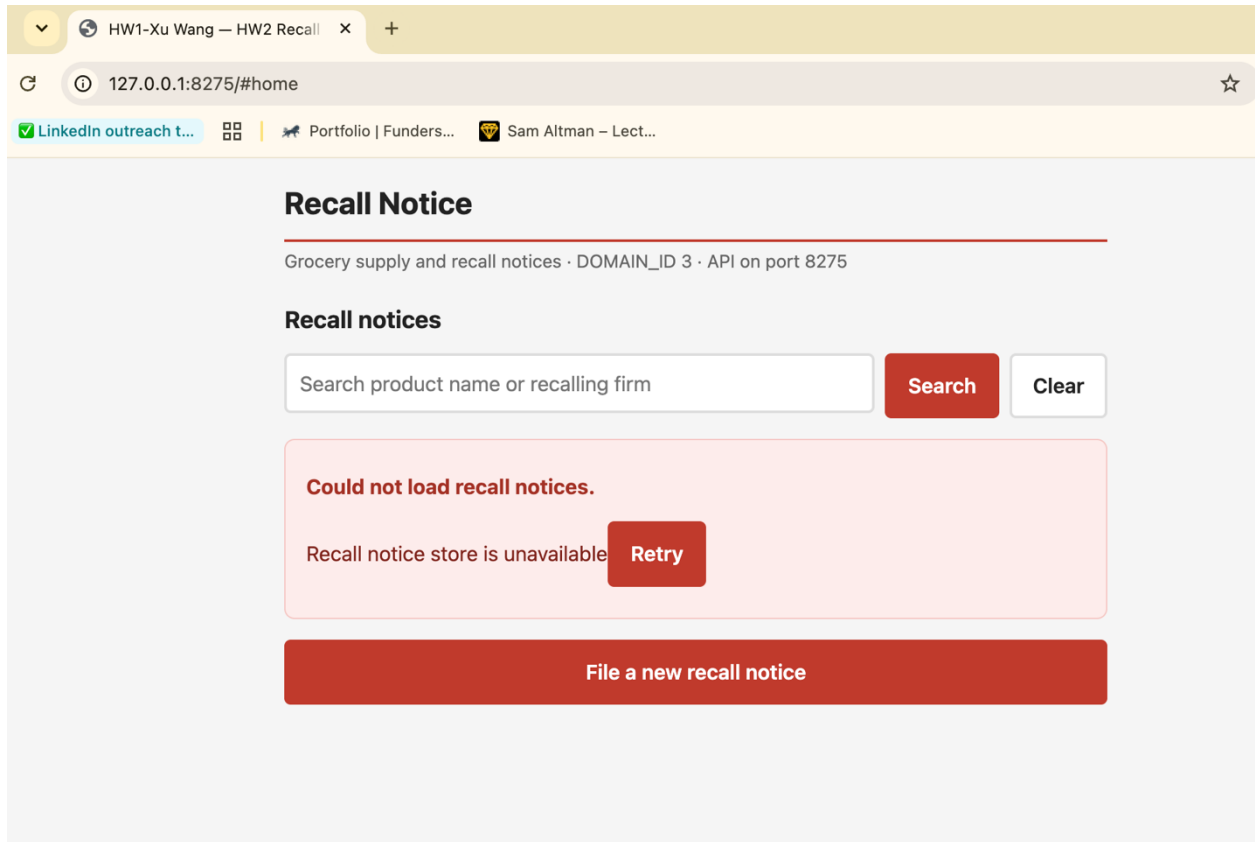
@media (prefers-reduced-motion: reduce) {
  .spinner { animation-duration: 2.4s; }
}

```

One thing changed from Homework 1. HW1 reported validation failures with `alert()`. Part 1 asks for visible error states, and an alert is not part of the page. It cannot be screenshotted in place and it disappears when you click it. The same two rules now mark the bad field inside the page. The rules themselves did not change, which is why `verify_hw01.py` still finds the 25-character check and the terms check.







Part 2 — FastAPI

code/api_server.py serves both the JSON API and the Part 1 page on PORT_BASE 8275, so the whole application is one process on one port. Records are stored in code/data/recall_notices.json. That file is created on first start from a seed file that is committed, so a fresh clone always starts with the same three notices.

The request model repeats the same rules the browser checks. A check in the browser is a convenience, not a guarantee. Anyone can POST to this endpoint without using the page at all.

code/api_server.py — the request model

```
class NoticeIn(BaseModel):
    """What the client sends on POST and PUT."""

    productName: str = Field(min_length=1)
    recallingFirm: str = Field(min_length=1)
    submitterEmail: str
    description: str
    category: Literal[CATEGORIES] # type: ignore[valid-type]
    agreedToTerms: bool
    submissionDate: Optional[str] = None

    @field_validator("submitterEmail")
    @classmethod
    def _email(cls, value: str) -> str:
        if not EMAIL_RE.match(value.strip()):
            raise ValueError("submitterEmail is not a valid email address")
        return value.strip()
```



```

@field_validator("description")
@classmethod
def _description(cls, value: str) -> str:
    # The same rule the browser enforces in app.js, enforced again here.
    # A client-side check is a convenience; it is not a guarantee, because
    # anyone can POST to this endpoint without going through the page.
    if len(value.strip()) <= 25:
        raise ValueError("description must be longer than 25 characters")
    return value

@field_validator("agreedToTerms")
@classmethod
def _terms(cls, value: bool) -> bool:
    if not value:
        raise ValueError("agreedToTerms must be true")
    return value

```

2.1 Add a new record

The user fills in the primary field (product name) and the secondary field (recalling firm), plus the other required fields. On success the server assigns the next id and the browser goes back to the home view with the updated list.

code/api_server.py

```

@app.post("/api/notices", response_model=Notice, status_code=201)
async def create_notice(payload: NoticeIn) -> dict:
    """Part 2.1 - add a new record and return it with its assigned id."""
    notices = load_notices()
    record = payload.model_dump()
    record["id"] = next_id(notices)
    record["submissionDate"] = payload.submissionDate or datetime.now(timezone.utc).isoformat()
    notices.append(record)
    save_notices(notices)
    print(f"[POST]    created notice {record['id']}: {record['productName']}")
    return record

```

code/web_application/app.js — POST, then back to the home view

```

const submitBtn = $("submitBtn");
const originalLabel = submitBtn.textContent;
submitBtn.disabled = true;
submitBtn.textContent = editingId === null ? "Submitting..." : "Saving...";

try {
    if (editingId === null) {
        // Part 2.1 - add a new record
        const created = await request(API, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(parsedDataWithDate),
        });
        showToast("Recall notice #" + created.id + " added.");
    } else {
        // Part 2.2 - update an existing record
        await request(API + "/" + editingId, {
            method: "PUT",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(parsedDataWithDate),
        });
        showToast("Recall notice #" + editingId + " updated.");
    }
    // Parts 2.1 and 2.2 - back to the home view with the updated list.
    goHome();
} catch (err) {
    showToast("Save failed: " + err.message, true);
} finally {
    submitBtn.disabled = false;
    submitBtn.textContent = originalLabel;
}
});

```

HW1-Xu Wang — HW2 Recall

127.0.0.1:8275/#new

✓ Claude

✓ LinkedIn outreach t...

Portfolio | Funders...

Sam Altman — Lect...

File a new recall notice

Product Name *

Creamy Peanut Milk

Recalling Firm *

Sunnyvale Food Inc

Submitter Email *

foodmilk@example.com

Recall Details *

The peanut milk are detected and there are unknown bacteria inside we need further test to check whether it is harmful to the potential customers who has allergic background. Will report more information when the results updates. Sorry for the mistake.

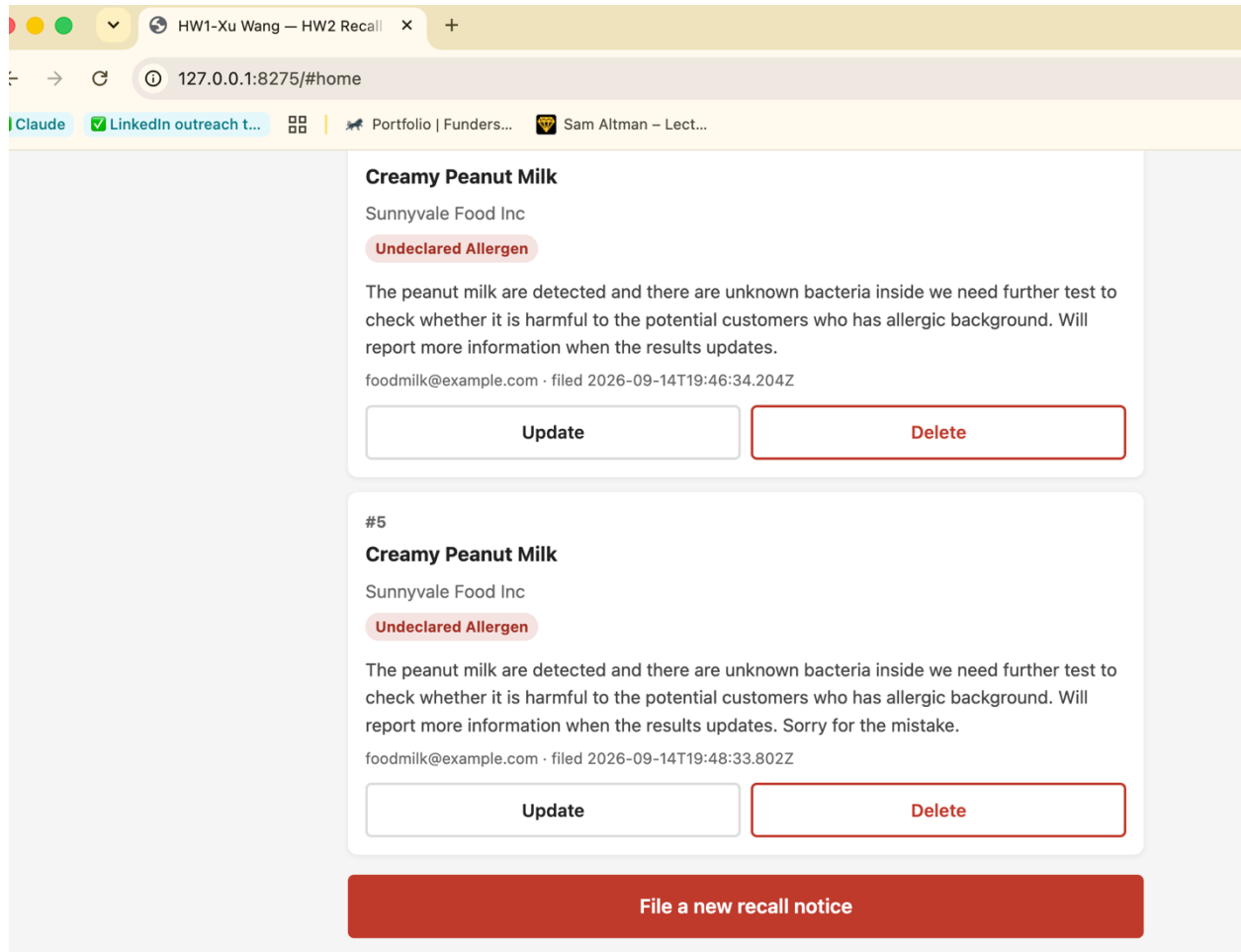
Reason for Recall *

Undeclared Allergen

☒ I agree to the terms and conditions.

Submit Recall Notice

Cancel



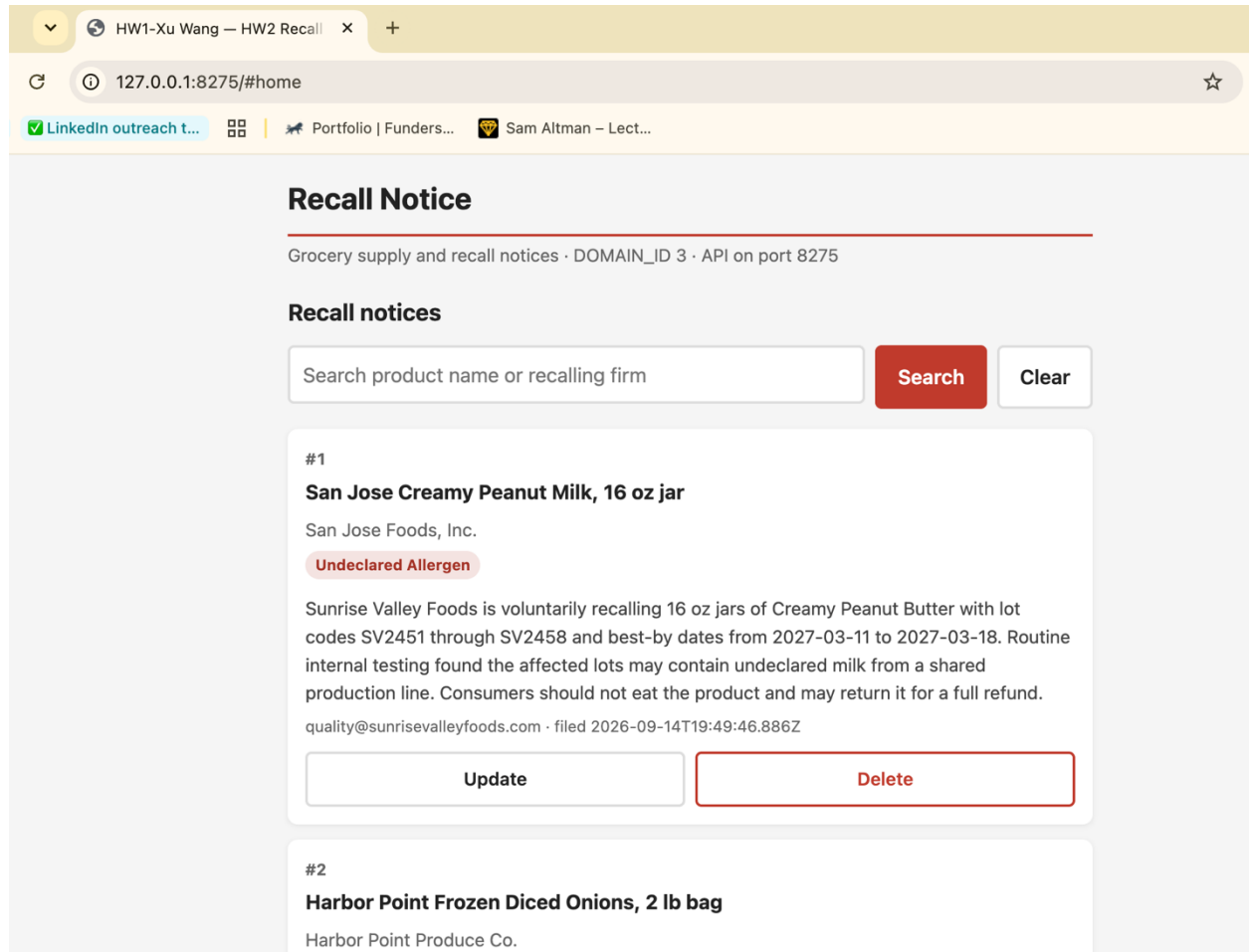
2.2 Update the record with ID 1

The same form is used for updating. It is filled in from GET `/api/notices/1` when the URL hash is `#edit/1`. The id cannot be edited, because the id is what says which record is being changed. `submissionDate` is kept if the client does not send a new one.

code/api_server.py

```
@app.put("/api/notices/{notice_id}", response_model=Notice)
async def update_notice(notice_id: int, payload: NoticeIn) -> dict:
    """Part 2.2 - replace the primary, secondary and remaining fields of a record.

    The id is not editable: it identifies the record being changed.
    """
    notices = load_notices()
    index = find_index(notices, notice_id)
    record = payload.model_dump()
    record["id"] = notice_id
    record["submissionDate"] = payload.submissionDate or notices[index]["submissionDate"]
    notices[index] = record
    save_notices(notices)
    print(f"[PUT] updated notice {notice_id}: {record['productName']}")
    return record
```

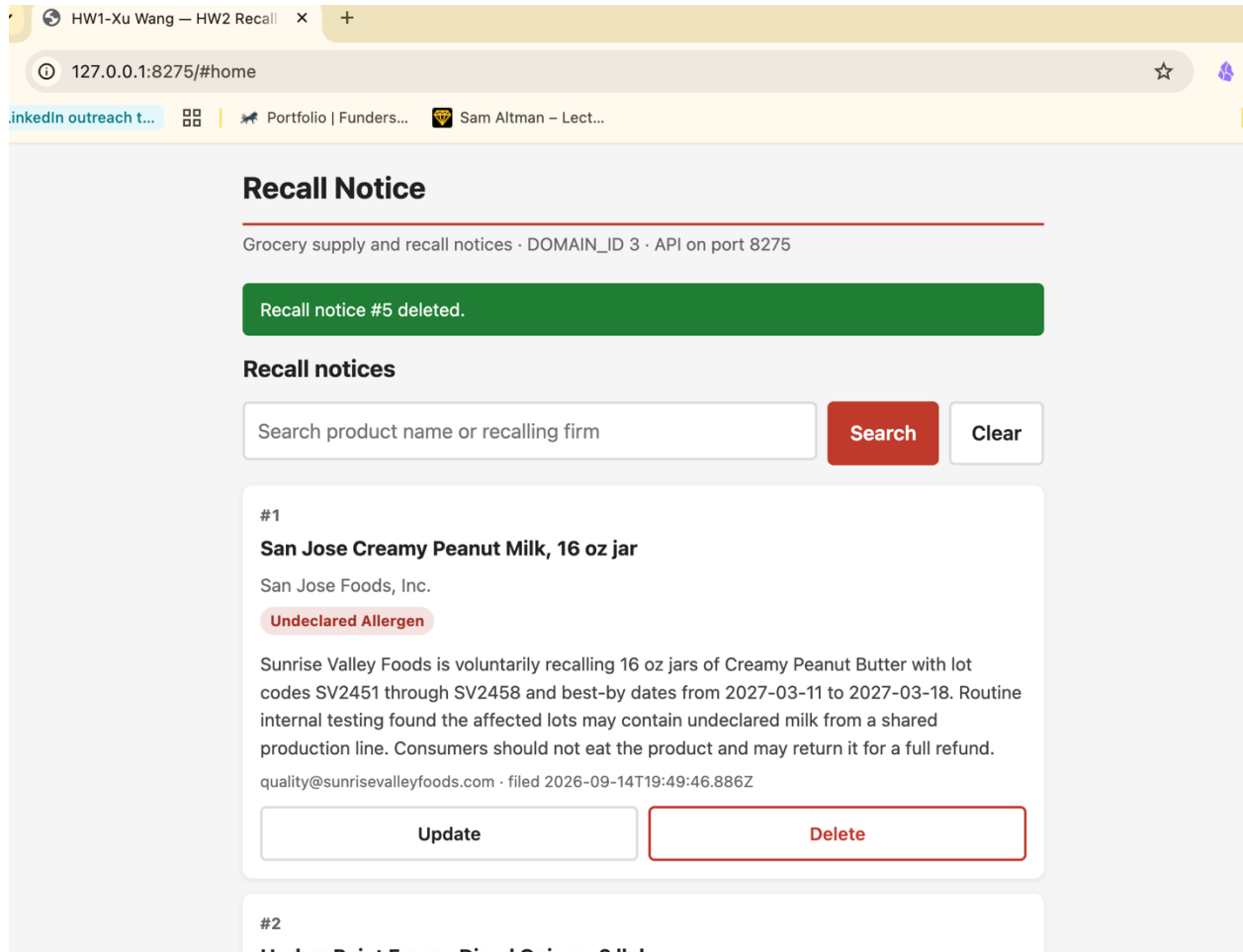


2.3 Delete the record with the highest ID

Deleting takes a second click instead of opening a confirm() dialog. That way the confirmation is part of the page and shows up in a screenshot. The server returns 204 with an empty body. The browser then reloads the list.

code/api_server.py

```
@app.delete("/api/notices/{notice_id}", status_code=204)
async def delete_notice(notice_id: int) -> Response:
    """Part 2.3 - remove a record."""
    notices = load_notices()
    index = find_index(notices, notice_id)
    removed = notices.pop(index)
    save_notices(notices)
    print(f"[DELETE] removed notice {notice_id}: {removed['productName']}")
    return Response(status_code=204)
```



2.4 Search by the primary or secondary field

The search term is sent to the server. It is not filtered in the browser. Matching ignores case and covers productName (primary) and recallingFirm (secondary). A search with no matches returns 200 and an empty list, not an error. That is why the page shows the empty state and not the error state.

code/api_server.py

```
@app.get("/api/notices", response_model=List[Notice])
async def list_notices(
    response: Response,
    q: Optional[str] = Query(default=None, description="match product name or recalling firm"),
) -> List[dict]:
    """Part 2.4 - list, optionally filtered by a search term.

    The term is matched case-insensitively against the primary field
    (productName) and the secondary field (recallingFirm).
    """
    delay_ms = int(os.getenv("DEMO_DELAY_MS", "0"))
    if delay_ms:
        await asyncio.sleep(delay_ms / 1000.0)

    if os.getenv("DEMO_FAIL_LIST") == "1":
        raise HTTPException(status_code=500, detail="Recall notice store is unavailable")

    response.headers["Cache-Control"] = "no-store"
    notices = load_notices()
```

```
    if q:
        needle = q.strip().lower()
        notices = [
            n
            for n in notices
            if needle in n["productName"].lower() or needle in n["recallingFirm"].lower()
        ]
    return notices
```

code/web_application/app.js

```
// Part 2.4 - search. The query is sent to the server, which matches it against
// the primary field (productName) and the secondary field (recallingFirm).
const loadNotices = async (query) => {
    showLoading();
    try {
        const url = query ? API + "?q=" + encodeURIComponent(query) : API;
        const notices = await request(url);
        if (notices.length === 0) {
            showEmpty(query);
            return;
        }
        hideState();
        renderNotices(notices);
    } catch (err) {
        showError(err.message);
    }
};
```

HW1-Xu Wang — HW2 Recall

127.0.0.1:8275/#home

ClaudeLinkedIn outreach t...Portfolio | Funders...Sam Altman – Lect...

ensions: iPhone SI375x 667Fit to windowNo throttling'Save-Data': defau

Recall notices

onion

SearchClear

#2

Harbor Point Frozen Diced Onions, 2 lb bag

Harbor Point Produce Co.

Bacterial Contamination

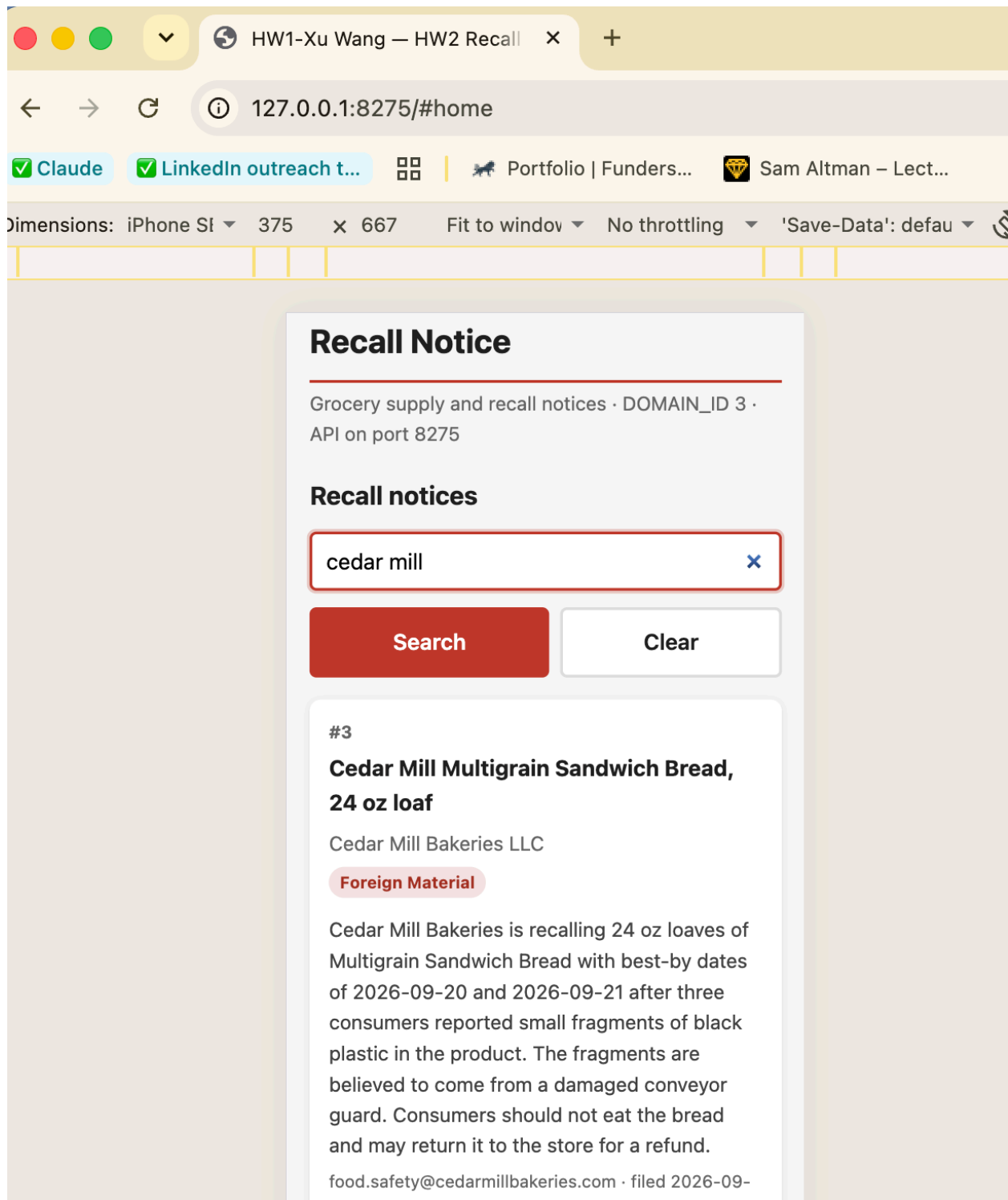
Harbor Point Produce is recalling 2 lb bags of Frozen Diced Onions with lot code HP-0918 after a supplier notified the company that the raw onions tested positive for Listeria monocytogenes. The product was distributed to foodservice customers in Washington and Idaho. Customers should discard the product and contact the firm for credit.

recalls@harborpointproduce.com · filed 2026-09-02T09:14:41.108Z

UpdateDelete

File a new recall notice

0.2 kB transferred ov

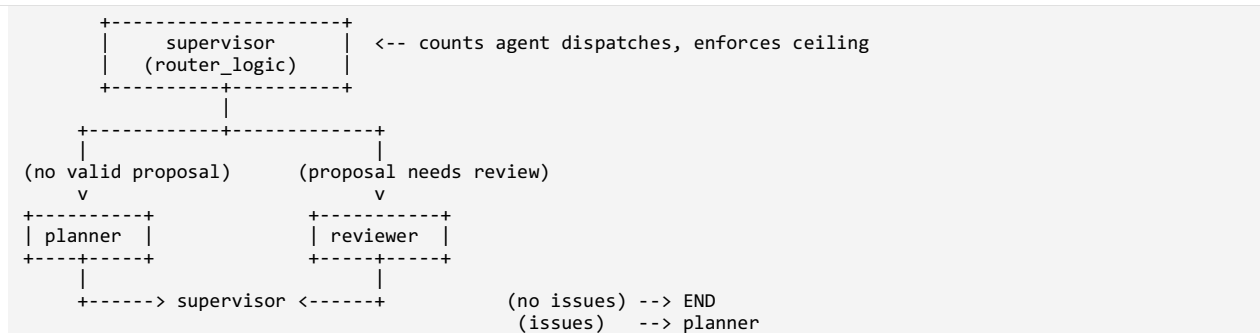


Part 3 — Stateful agent graph

Homework 1 ran Planner, then Reviewer, then Finalizer in a straight line. A line cannot go back. So if the Reviewer disagreed with the Planner it had no way to ask for a correction. It could only overwrite the

answer itself. Homework 2 makes the same two agents into nodes of a LangGraph StateGraph, with a supervisor that decides after every step where control goes next.

The flow implemented in `code/agent_graph.py`



Every model call still goes through `src/model_client.py`, the adapter from Homework 1. `agent_graph.py` does not import `ollama` and does not import `langchain`. `verify_hw02.py` checks this by reading the import statements rather than by searching the text, so the docstring is allowed to mention those names.

Step 2 — the shared state

`AgentState` is the memory every node reads from and writes to. The first block is the state the assignment suggests. The second block is what Part 4 needs in order to classify a run after it finishes.

`code/agent_graph.py`

```

class AgentState(TypedDict, total=False):
    """The memory every node reads from and writes to.

    The first block is the assignment's suggested state. The second block is
    what Part 4 needs in order to classify a run afterwards.
    """

    # --- inputs, present before the graph starts ---
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any  # the ModelClient instance

    # --- agent outputs ---
    planner_proposal: Optional[Dict[str, Any]]
    reviewer_feedback: Optional[Dict[str, Any]]
    turn_count: int

    # --- loop safety and bookkeeping (Part 4) ---
    turn_ceiling: int
    validation_error: Optional[str]
    planner_attempts: int
    reviewer_attempts: int
    status: str  # running | complete | abandoned
    force_reviewer_issues: bool
    history: List[Dict[str, Any]]
  
```

A node never changes this dictionary directly. It returns a dictionary with only the keys it wants to change, and LangGraph merges that into the state. That is why every node below ends with a return instead of an assignment.

Steps 3 and 4 — the agents and the supervisor

The supervisor does no work itself. Its only job is to count and to stop. The counter is in one place only. That is what makes the ceiling a guarantee instead of a convention, because there is exactly one line in the program where a run can be ended.

One turn is one dispatch to an agent. A run that is right the first time costs two turns: Planner once, Reviewer once. The supervisor visits that go straight to END are not counted, because no agent ran. This matters for Part 4.4. If the counter went up on every supervisor visit, a clean run would cost three turns, and a ceiling of 2 could never finish any run at all, not even a correct one.

The supervisor and the router call the same function, so the counter and the route cannot disagree about what happens next.

code/agent_graph.py

```
def next_worker(state: AgentState) -> Optional[str]:
    """Which agent still has work to do, or None if the run is over.

    Order matters. Each branch answers one question, and the first one that
    applies wins:
    1. is the run already over, either finished or abandoned?
    2. is there a proposal that passed validation?
    3. has that proposal been reviewed yet?
    4. did the review find something wrong?

    Both the supervisor node and the routing function read this one predicate,
    so the counter and the route can never disagree about what happens next.
    """
    if state.get("status") in ("complete", "abandoned"):
        return None

    if state.get("validation_error") or state.get("planner_proposal") is None:
        return "planner"

    feedback = state.get("reviewer_feedback")
    if feedback is None:
        return "reviewer"

    if feedback.get("has_issues"):
        return "planner"

    return None
```

code/agent_graph.py

```
def supervisor_node(state: AgentState) -> Dict[str, Any]:
    """Counts turns and stops the graph when the ceiling is reached.

    This node does no work of its own. Keeping the counter here, rather than in
    the agents, means there is exactly one place where a run can be declared
    over, which is what makes the ceiling a guarantee instead of a convention.

    One turn is one dispatch to an agent. A run that is right the first time
    therefore costs two turns: Planner once, Reviewer once. The supervisor
    visits that route straight to END are not counted, because no agent ran.
    """
    ceiling = state.get("turn_ceiling", DEFAULT_CEILING)
```

```

    pending = next_worker(state)

    if pending is None:
        print(f"---NODE: Supervisor (no work left, {state.get('turn_count', 0)} turns used) ---")
        return {}

    turn = state.get("turn_count", 0) + 1
    print(f"---NODE: Supervisor (turn {turn}/{ceiling} -> {pending}) ---")

    update: Dict[str, Any] = {"turn_count": turn}
    if turn > ceiling:
        print(f"    turn ceiling {ceiling} reached; abandoning the run")
        update["status"] = "abandoned"
    return update

def router_logic(state: AgentState) -> str:
    """Reads the state and names the next node. Writes nothing."""
    worker = next_worker(state)
    return worker if worker is not None else END

```

Step 5 — assembling the graph

code/agent_graph.py

```

def build_graph():
    graph = StateGraph(AgentState)

    graph.add_node("supervisor", supervisor_node)
    graph.add_node("planner", planner_node)
    graph.add_node("reviewer", reviewer_node)

    graph.set_entry_point("supervisor")

    # The supervisor is the only node with a choice to make.
    graph.add_conditional_edges(
        "supervisor",
        router_logic,
        {"planner": "planner", "reviewer": "reviewer", END: END},
    )

    # Both workers hand control straight back, so the turn counter and the
    # ceiling are applied after every single step.
    graph.add_edge("planner", "supervisor")
    graph.add_edge("reviewer", "supervisor")

    return graph.compile()

```

Both workers have a plain edge back to the supervisor instead of a conditional edge of their own. Only the supervisor has a choice to make, so only the supervisor gets conditional edges. This means the turn counter and the ceiling are applied after every single step.

Step 6 — running and testing

Invoking with `.stream()` prints each node's update as it happens rather than only the final state.

code/agent_graph.py

```

if stream:
    # Part 3 Step 6 - .stream() shows the output of each step as it happens.
    for chunk in app.stream(state, config=config):
        for node_name, update in chunk.items():
            # A node that changed nothing yields None here. The supervisor

```

```
# does that on its last visit, when no agent still owes work.
if not update:
    continue
final.update(update)
keys = [k for k in update if k not in ("history", "llm")]
print(f"    [stream] {node_name} updated: {' ', '.join(keys)}")
else:
    final = dict(app.invoke(state, config=config))
```

Before any run against the local model, the control flow of the graph is checked offline.

code/graph_offline_test.py replaces ModelClient with a fake that returns prepared answers. Thirteen behaviours are checked in about one second without Ollama: a clean run, a schema rejection followed by a retry, a Reviewer sending work back, a Reviewer that always objects, and the same ceiling on both the .stream() and the .invoke() path. This test found two real problems before the experiment was run. Both are described in AI_USE.md.

```

[(data260) Sunny@MacBook-Pro-7 data260-9275 % make reset-data ]
rm -f code/data/recall_notices.json
record store cleared; it is recreated from the seed file on next start
[(data260) Sunny@MacBook-Pro-7 data260-9275 % python code/graph_offline_test.py ]
---NODE: Supervisor (turn 1/10 -> planner) ---
---NODE: Planner ---
    accepted: {"tags": ["peanut butter", "undeclared milk", "product recall"], "
summary": "Sunrise Valley recalls peanut butter jars that may contain undeclared
milk."}
---NODE: Supervisor (turn 2/10 -> reviewer) ---
---NODE: Reviewer ---
    has_issues=False: nothing needed changing
---NODE: Supervisor (no work left, 2 turns used) ---
PASS valid first attempt outcome='valid first attempt'
turns=2 planner=1 reviewer=1
---NODE: Supervisor (turn 1/10 -> planner) ---
---NODE: Planner ---
    rejected by schema: summary: Value error, summary is 40 words; at most 25 ar
e allowed
---NODE: Supervisor (turn 2/10 -> planner) ---
---NODE: Planner ---
    accepted: {"tags": ["peanut butter", "undeclared milk", "product recall"], "
summary": "Sunrise Valley recalls peanut butter jars that may contain undeclared
milk."}
---NODE: Supervisor (turn 3/10 -> reviewer) ---
---NODE: Reviewer ---
    has_issues=False: nothing needed changing
---NODE: Supervisor (no work left, 3 turns used) ---
PASS schema reject -> 1 retry outcome='valid after 1 retry'
turns=3 planner=2 reviewer=1
    validation error fed back: summary: Value error, summary is 40 words; at m
ost 25 are allowed
---NODE: Supervisor (turn 1/10 -> planner) ---
---NODE: Planner ---
    rejected by schema: tags: Value error, tag 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaa' is 45 characters; each tag must be 3-30 characters
---NODE: Supervisor (turn 2/10 -> planner) ---
---NODE: Planner ---
    accepted: {"tags": ["peanut butter", "undeclared milk", "product recall"], "
summary": "Sunrise Valley recalls peanut butter jars that may contain undeclared
milk."}
---NODE: Supervisor (turn 3/10 -> reviewer) ---
---NODE: Reviewer ---
    has_issues=False: nothing needed changing
---NODE: Supervisor (no work left, 3 turns used) ---
PASS tag >30 chars rejected outcome='valid after 1 retry'
turns=3 planner=2 reviewer=1
    validation error fed back: tags: Value error, tag 'aaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaa'
---NODE: Supervisor (turn 1/10 -> planner) ---
---NODE: Planner ---

```



```

    [stream] supervisor updated: turn_count
---NODE: Reviewer ---
    has_issues=False: nothing needed changing
    [stream] reviewer updated: reviewer_feedback, reviewer_attempts, status
---NODE: Supervisor (no work left, 3 turns used) ---
PASS  stream() path                               outcome='valid after 1 retry'
turns=3 planner=2 reviewer=1
---NODE: Supervisor (turn 1/2 -> planner) ---
---NODE: Planner ---
    accepted: {"tags": ["peanut butter", "undeclared milk", "product recall"], "
summary": "Sunrise Valley recalls peanut butter jars that may contain undeclared
milk."}
---NODE: Supervisor (turn 2/2 -> reviewer) ---
---NODE: Reviewer ---
    has_issues=False: nothing needed changing
---NODE: Supervisor (no work left, 2 turns used) ---
PASS  clean run under ceiling 2                     outcome='valid first attempt'
turns=2 planner=1 reviewer=1
---NODE: Supervisor (turn 1/2 -> planner) ---
---NODE: Planner ---
    rejected by schema: summary: Value error, summary is 40 words; at most 25 ar
e allowed
---NODE: Supervisor (turn 2/2 -> planner) ---
---NODE: Planner ---
    accepted: {"tags": ["peanut butter", "undeclared milk", "product recall"], "
summary": "Sunrise Valley recalls peanut butter jars that may contain undeclared
milk."}
---NODE: Supervisor (turn 3/2 -> reviewer) ---
    turn ceiling 2 reached; abandoning the run
PASS  one retry under ceiling 2                     outcome='hit turn ceiling'
turns=3 planner=2 reviewer=0
---NODE: Supervisor (turn 1/10 -> planner) ---
---NODE: Planner ---
    rejected by schema: summary: Value error, summary is 40 words; at most 25 ar
e allowed
---NODE: Supervisor (turn 2/10 -> planner) ---
---NODE: Planner ---
    accepted: {"tags": ["peanut butter", "undeclared milk", "product recall"], "
summary": "Sunrise Valley recalls peanut butter jars that may contain undeclared
milk."}
---NODE: Supervisor (turn 3/10 -> reviewer) ---
---NODE: Reviewer ---
    has_issues=False: nothing needed changing
---NODE: Supervisor (no work left, 3 turns used) ---
PASS  one retry under ceiling 10                    outcome='valid after 1 retry'
turns=3 planner=2 reviewer=1

recursion limit headroom check: ceiling 10 forced = 30 steps allowed

13/13 behaviour tests passed

```

```

[(data260) Sunny@MacBook-Pro-7 data260-9275 % make run-graph
cd code && python -u agent_graph.py 2>&1 | tee -a ../reports/hw02/RUN_LOG.txt
---NODE: Supervisor (turn 1/10 -> planner) ---
    [stream] supervisor updated: turn_count
---NODE: Planner ---
    accepted: {"tags": ["peanut butter", "milk allergy", "product recall"], "summary": "sunrise valley foods recalls creamy peanut butter due to potential milk contamination affecting specific lots and regions."}
    [stream] planner updated: planner_proposal, validation_error, reviewer_feedback, planner_attempts
---NODE: Supervisor (turn 2/10 -> reviewer) ---
    [stream] supervisor updated: turn_count
---NODE: Reviewer ---
    has_issues=False: nothing is wrong
    [stream] reviewer updated: reviewer_feedback, reviewer_attempts, status
---NODE: Supervisor (no work left, 2 turns used) ---

=====
RESULT
=====
outcome          : valid first attempt
status           : complete
turns used       : 2 / 10
planner attempts : 1
reviewer attempts: 1
model calls      : 2
total latency    : 415703 ms

{
  "tags": [
    "peanut butter",
    "milk allergy",
    "product recall"
  ],
  "summary": "sunrise valley foods recalls creamy peanut butter due to potential milk contamination affecting specific lots and regions."
}

```

The correction loop

The assignment asks for the Reviewer to be changed so that it always returns an issue, so the graph can be watched sending the task back to the Planner. I did this with a command-line flag instead of editing reviewer_node. If I edit the code by hand I have to remember to undo it, and any run done before undoing it is wrong without any warning. A flag can be run again by anyone and it leaves the normal path alone.

code/agent_graph.py — reviewer_node

```

def reviewer_node(state: AgentState) -> Dict[str, Any]:
    """Check the validated proposal against the source text.

    Part 3 Step 6 asks for the correction loop to be demonstrated by making the
    Reviewer always complain. That is the --force-reviewer-issues flag rather
    than an edit to this function, so the loop test is reproducible and the
    normal path is never left modified by accident.

```

```

"""
print("---NODE: Reviewer ---")
attempts = state.get("reviewer_attempts", 0) + 1
history = list(state.get("history", []))

if state.get("force_reviewer_issues"):
    feedback = {
        "has_issues": True,
        "issues": "forced rejection for the loop test (--force-reviewer-issues)",
        "forced": True,
    }
    history.append(
        {"node": "reviewer", "attempt": attempts, "turn": state.get("turn_count", 0),
         "forced": True, "has_issues": True}
    )
    print("    forced issue; sending the task back to the Planner")
    return {"reviewer_feedback": feedback, "reviewer_attempts": attempts, "history": history}

client: ModelClient = state["llm"]
proposal = state.get("planner_proposal") or {}

response = client.complete(
    messages=[
        {"role": "system", "content": REVIEWER_SYSTEM},
        {
            "role": "user",

```



```
(data260) Sunny@MacBook-Pro-7 data260-9275 % cd code && python -u agent_graph.py
--force-reviewer-issues --ceiling 4 2>&1 | tee -a ../reports/hw02/RUN_LOG.txt;
cd ..
```

```
---NODE: Supervisor (turn 1/4 -> planner) ---
```

```
  [stream] supervisor updated: turn_count
```

```
---NODE: Planner ---
```

```
  accepted: {"tags": ["peanut butter", "milk allergy", "product recall"], "summary": "sunrise valley foods recalls creamy peanut butter due to potential milk contamination affecting specific lots and regions."}
```

```
  [stream] planner updated: planner_proposal, validation_error, reviewer_feedback, planner_attempts
```

```
---NODE: Supervisor (turn 2/4 -> reviewer) ---
```

```
  [stream] supervisor updated: turn_count
```

```
---NODE: Reviewer ---
```

```
  forced issue; sending the task back to the Planner
```

```
  [stream] reviewer updated: reviewer_feedback, reviewer_attempts
```

```
---NODE: Supervisor (turn 3/4 -> planner) ---
```

```
  [stream] supervisor updated: turn_count
```

```
---NODE: Planner ---
```

```
  accepted: {"tags": ["peanut butter", "milk contamination", "product recall"], "summary": "sunrise valley foods recalls peanut butter due to milk contamination in specific lots distributed to california nevada and oregon."}
```

```
  [stream] planner updated: planner_proposal, validation_error, reviewer_feedback, planner_attempts
```

```
---NODE: Supervisor (turn 4/4 -> reviewer) ---
```

```
  [stream] supervisor updated: turn_count
```

```
---NODE: Reviewer ---
```

```
  forced issue; sending the task back to the Planner
```

```
  [stream] reviewer updated: reviewer_feedback, reviewer_attempts
```

```
---NODE: Supervisor (turn 5/4 -> planner) ---
```

```
  turn ceiling 4 reached; abandoning the run
```

```
  [stream] supervisor updated: turn_count, status
```

```
=====
RESULT
```

```
outcome      : hit turn ceiling
```

```
status       : abandoned
```

```
turns used   : 5 / 4
```

```
planner attempts : 2
```

```
reviewer attempts: 2
```

```
model calls   : 2
```

```
total latency : 60812 ms
```

```
{
  "tags": [
```

Part 4 — Output schema and loop safety

4.1 Validating the Planner's output with Pydantic

The contract is exactly three string tags, each 3–30 characters, and a summary of at most 25 words.

code/agent_graph.py

```
class PlannerOutput(BaseModel):
    """What the Planner must produce for the run to continue.

    The JSON *shape* is already enforced by Ollama's `format` parameter. What
    that cannot express is the semantic part of the contract: how long a tag
    may be and how many words a summary may contain. Those are the two rules
    below, and they are the ones that actually fail in practice.
    """

    tags: List[str] = Field(min_length=TAG_COUNT, max_length=TAG_COUNT)
    summary: str

    @field_validator("tags")
    @classmethod
    def _tag_lengths(cls, tags: List[str]) -> List[str]:
        for tag in tags:
            if not isinstance(tag, str):
                raise ValueError(f"tag {tag!r} is not a string")
            if not (TAG_MIN_CHARS <= len(tag) <= TAG_MAX_CHARS):
                raise ValueError(
                    f"tag {tag!r} is {len(tag)} characters; each tag must be "
                    f"{TAG_MIN_CHARS}-{TAG_MAX_CHARS} characters"
                )
        return tags

    @field_validator("summary")
    @classmethod
    def _summary_words(cls, summary: str) -> str:
        words = len(summary.split())
        if words > MAX_SUMMARY_WORDS:
            raise ValueError(
                f"summary is {words} words; at most {MAX_SUMMARY_WORDS} are allowed"
            )
        if words == 0:
            raise ValueError("summary is empty")
        return summary
```

This layer is not a duplicate of the JSON schema that is already sent to Ollama. The format parameter guarantees that the reply parses and has exactly three string tags. It cannot say “each tag is 3 to 30 characters” or “the summary is at most 25 words”. Every rejection recorded in this homework was a length violation that the JSON schema had already accepted.

4.2 Feeding the validation error back

A rejected proposal never reaches the Reviewer. The Planner is told what was wrong and tries again. The feedback is the only thing that changes between attempts.

code/agent_graph.py — the gate

```
try:
    validated = PlannerOutput(**raw)
except ValidationError as err:
    message = format_validation_error(err)
    entry.update({"valid": False, "error": message, "raw": raw})
    history.append(entry)
    print(f"    rejected by schema: {message}")
    return {
        "planner_proposal": None,
```

```

        "validation_error": message,
        "reviewer_feedback": None,
        "planner_attempts": attempts,
        "history": history,
    }

    proposal = validated.model_dump()

```

code/agent_graph.py — the feedback on the next attempt

```

attempts = state.get("planner_attempts", 0) + 1
history = list(state.get("history", []))

user_parts = [build_input_block(state)]

if state.get("validation_error"):
    # Part 4.2 - feed the validation error back to the Planner.
    user_parts.append(
        "YOUR PREVIOUS ANSWER WAS REJECTED BY THE OUTPUT VALIDATOR:\n"
        f"{state['validation_error']}\n"
        "Fix exactly that problem and answer again."
    )
elif state.get("reviewer_feedback", {}) and state["reviewer_feedback"].get("has_issues"):
    user_parts.append(
        "A REVIEWER REJECTED YOUR PREVIOUS ANSWER:\n"
        f"previous: {json.dumps(state.get('planner_proposal'), ensure_ascii=False)}\n"
        f"reviewer said: {state['reviewer_feedback'].get('issues', '')}\n"
        "Address that and answer again."
    )

```

code/agent_graph.py — how a finished run is classified

```

def classify(state: Dict[str, Any]) -> str:
    """Part 4.3 - the four outcome buckets."""
    if state.get("status") != "complete":
        return "hit turn ceiling"
    retries = max(0, int(state.get("planner_attempts", 1)) - 1)
    if retries == 0:
        return "valid first attempt"
    if retries == 1:
        return "valid after 1 retry"
    return "valid after 2+ retries"

```

4.3 Thirty runs on one frozen input

The input was written to reports/hw02/cases/schema_input.json before the experiment and was not edited afterwards. It is the same recall notice used in the Homework 1 non-determinism experiment, so the two homeworks measure different things on the same text. Model qwen3:8b, temperature 0.0, turn ceiling 10.

Outcome over 30 runs	Count	Mean latency (ms)
Valid first attempt	30	17,594
Valid after 1 retry	0	—
Valid after 2+ retries	0	—
Hit turn ceiling	0	—

Latency p50 15,224 ms, p95 33,030 ms, min 9,027 ms, max 53,767 ms. Every run cost exactly 2 turns and 2 model calls.

The retry path never fired. This is a result, not a missing one. The margins show why. Summaries came in at 17 to 18 words against a limit of 25, and tags at 12 to 14 characters against a limit of 30. The gate was nowhere near binding on well-formed input.

```
[(data260) Sunny@MacBook-Pro-7 data260-9275 % cd code && python graph_experiment_
runner.py --summarize; cd ..
```

```
Part 4.3 - 30 runs on the frozen input, ceiling 10
Outcome over 30 runs      Count      Mean latency (ms)
valid first attempt       30          17594.4
valid after 1 retry        0              0
valid after 2+ retries     0              0
hit turn ceiling          0              0
```

Part 4.4 - turn ceiling comparison, same input and model settings

```
Metric                  ceiling 2    ceiling 10
runs                    20          20
completion rate         100%         100%
mean latency (ms)       13941.3      19167.6
mean latency, completed (ms) 13941.3      19167.6
mean turns used         2            2
mean planner attempts   1            1
mean model calls        2            2
```

Part 4.5 - adversarial input, 5 runs, ceiling 10

```
valid first attempt      0
valid after 1 retry      0
valid after 2+ retries   0
hit turn ceiling         5
completion rate          0%
```

```
raw runs   : /Users/Sunny/Documents/data260-9275/reports/hw02/raw/graph_runs.js
nl
flat table : /Users/Sunny/Documents/data260-9275/reports/hw02/raw/graph_runs.csv
aggregates : /Users/Sunny/Documents/data260-9275/reports/hw02/raw/hw02_metrics.j
son
```

4.4 Turn ceiling 2 versus 10

Same frozen input, same model, same temperature, 20 runs each.

Metric	Ceiling 2	Ceiling 10
Completion rate	100% (20/20)	100% (20/20)
Mean latency (ms)	13,941	19,168
Median latency (ms)	11,163	14,618
p95 latency (ms)	19,343	35,838
Mean turns used	2.0	2.0
Mean planner attempts	1.0	1.0
Mean model calls	2.0	2.0

Metric	Ceiling 2	Ceiling 10
Input / output tokens	12,760 / 1,300	12,760 / 1,300

The latency difference is not caused by the ceiling. Both arms ran exactly two turns and two model calls per run, and used the same number of tokens down to the byte. A ceiling only does something when a run tries to go past it, and no run did. The 5.2 second gap in the means is machine noise on an 8 GB laptop. The same effect was recorded in Homework 1, where runs 10 to 15 of one batch slowed from 12 s to 20 to 36 s under memory pressure while the work being done did not change. Reading this gap as “ceiling 2 is faster” would be a misreading of this data.

Which one to deploy: ceiling 2

- On well-formed input the two are the same: same completion rate, same turns, same token cost.
- They differ only on input that fails, and there the difference is the cost of failing. Under ceiling 10 the adversarial case took 11 turns, 10 model calls and 95 seconds to reach the same outcome that ceiling 2 reaches in 3 turns and 2 model calls. I measured this directly on the same input. Ceiling 2 abandoned it in 38.1 s with a warm model. The first run of the session, which had to load the model, took 67.5 s.
- So the choice is not between succeeding and failing. It is between failing fast and failing slow. Five times the model calls on every bad input buys nothing that this data can show.

There is a limit to this recommendation. The first-attempt rate was 100% over 30 runs, so this experiment never saw a run that a retry would have saved. It shows that retries are not needed here. It does not show that they are never needed. Ceiling 2 allows exactly one Planner attempt and one review. If some later input turns out to need one correction, ceiling 2 would throw away work that ceiling 3 or 4 would keep. Before generalising, 4.3 should be repeated on more inputs.

4.5 The adversarial input

reports/hw02/cases/adversarial_input.json looks like a real recall notice. What makes it hard is deliberate. The important product terms are long hyphenated words. Hydrolyzed-Vegetable-Protein-Seasoning-Blend is 44 characters. The notice also carries three separate hazards, two firms, four lot ranges and six states, so any summary that stays true to the content runs past 25 words.

Outcome over 5 runs	Count	Mean cost
Valid first attempt	0	—
Valid after 1 retry	0	—
Valid after 2+ retries	0	—
Hit turn ceiling	5	11 turns, 10 calls, 95,057 ms

Why it causes trouble

Not randomness. All five runs produced the same trace, which is what made the cause findable. Run 1, shortened:

Step	What happened
P1	tags [hydrolyzedvegetableprotein, listeriamonocytogenes, stainlesssteelwire] — VALID
R1	"stainlesssteelwire is not specific enough, should be stainlesssteelwirefragments"
P2	complies — VALID
R2	"hydrolyzedvegetableprotein is not specific enough"
P3	lengthens to hydrolyzedvegetableproteinseasoning, 35 chars — REJECTED by the validator
P4	backs off to 26 chars — VALID
R3	"should be hydrolyzed-vegetable-protein-seasoning-blend" — that string is 44 chars
P5	complies — REJECTED by the validator
P6	backs off again — VALID
R4	"product-recall is too generic" ... and so on until turn 11

The Planner is not failing. It produced a valid proposal on four of its six attempts. The run never ends because the Reviewer keeps asking for a tag that the validator does not allow. REVIEWER_SYSTEM asks for tags that are specific rather than generic and says nothing about the 3 to 30 character limit. For a product whose specific name is 44 characters, those two requirements contradict each other. No output can satisfy both.

This is a livelock, not a deadlock. Work is happening on every step, but no step gets the run closer to finishing. The turn ceiling is the only thing that stops it. Without it, this input would loop forever on a notice that a person would call ordinary.

One fix

Give the Reviewer the same output contract that the validator enforces. REVIEWER_SYSTEM should state the 3 to 30 character tag limit and the 25 word summary limit, and should reject a proposal only for a problem that can be fixed inside those limits. Then the contradiction cannot be written down. A Reviewer that knows about the 30 character limit cannot ask for a 44 character tag.

That is one change to a prompt and it deals with the cause. Raising the ceiling would only make the loop longer. Lowering it would hide the loop without explaining it. A stronger version of the same fix is to check the Reviewer's suggestion against PlannerOutput before it is sent back, so a suggestion that could not pass never reaches the Planner. This is the same idea as the deterministic Finalizer in Homework 1: put the hard contract in code, not in a prompt.

Two findings worth recording

Refactoring the Reviewer changed the published output

Same input, same model, same temperature. Homework 1 published [peanut butter, milk contamination, product recall]. Homework 2 publishes [peanut butter, milk allergy, product recall] in all 70 frozen-input runs.

Nothing about the model changed. In Homework 1 the Reviewer returned corrected tags and rewrote milk allergy into milk contamination in 37 of 40 runs. In Homework 2 the Reviewer only judges. It returns `has_issues` and a sentence, and it reported no issue in 70 of 70 runs. So the wording the Planner produced is what gets published.

This matters because of which field it is. The Homework 1 analysis picked out this exact tag as the safety-relevant one. If the tag decides which subscribers get an allergen alert, milk allergy and milk contamination go to different lists. I changed the architecture for control-flow reasons and it moved a safety-critical value with no warning.

Temperature 0.0 is still not a determinism guarantee

Of 70 frozen-input runs, 69 produced the same summary. The one that differed was the first run of the session, on a cold model. Tag sets were identical in all 70. This matches the Homework 1 finding on a larger sample. What makes an output reproducible is constraining its shape. Short fields with limits are much more stable than free prose.

Self-check

`make verify-hw02` starts the FastAPI server on `PORT_BASE` against a temporary record store, so the check never touches `code/data/recall_notices.json` or any application file. It then does create, update, delete and search over HTTP, runs the offline graph test, runs one live graph with a 240-second timeout, and re-checks the recorded Part 4 data against the output contract.

Every check is behavioural. None of them compares model output to a fixed string, because the model does not say the same thing twice. They ask questions that can be counted instead. Exactly three tags? Every tag 3 to 30 characters? Did the request succeed? Did the process exit? Results are written to `reports/hw02/verification.json`.

```
[(data260) Sunny@MacBook-Pro-7 data260-9275 % make verify-hw02
python verify_hw02.py
```

```
80/81 checks passed -> PASS
```

```
optional failures:
```

```
  warn file.reports/hw02/report.pdf export the write-up to PDF before tagging
wrote reports/hw02/verification.json
```